
Software Requirements Specification

for

Go cheap

Version 1.0

Prepared by

- ❖ Beshoy Karam (manager)
- ❖ Ahd Montaser
- ❖ Kerolous Samir
- ❖ Ahmed Altigany
- ❖ Sherif Abd Almoaty

Submitted to

Dr. Wesam Ahmed.

Faculty of computers and artificial intelligence

Hurghada University

Table of Contents

1. Introduction	1
1.1 Goals and Objectives	1
1.2 Scope	1
1.3 Constraints	1
1.4 References	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	3
2.5 Design and Implementation Constraints	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	4
3.3 Software Interfaces	4
3.4 Communications Interfaces	5
4. System Features	5
4.1 Price Comparator	5
4.2 User Interface	6
5. Other Nonfunctional Requirements	6
5.1 Performance Requirements	6
5.2 Safety Requirements	7
5.3 Security Requirements	7
5.4 Software Quality Attributes	7
5.5 Business Rules	7
6. Other Requirements	8
7. Database Diagram	9
7.1 ERD Diagram	9
7.2 Relation Schema Diagram	9
8. System modeling	10
8.1 Context Diagram	10
8.2 Process perspective Diagram	11
8.3 Use case Diagram	12
8.4 Tables of use cases	12
8.5 Class Diagram	17

1. Introduction

1.1 Goals and Objectives

- To develop a web-based system that enables users to compare ride prices across multiple transportation companies such as Uber, Didi, and On Drive.
- To allow users to input pick-up and drop-off locations and receive a list of available ride prices.
- Enable users to select a ride and redirect them to the chosen company's application.
- Help users save time and reduce transportation costs.

1.2 Scope

- Web application for comparing ride-hailing prices.
- Frontend developed using HTML, CSS, and Bootstrap.
- Backend implemented using Django.
- Use of mock data to simulate ride fares where real APIs are unavailable.
- No in-system booking or payment processing.

1.3 Constraints

- Limited access to real-time fare data due to the unavailability of public APIs from some ride-hailing companies.
- Dependence on mock data to simulate ride prices for comparison purposes.
- The system requires a stable internet connection to function correctly.

1.4 References

- **Django Framework:** <https://www.djangoproject.com/>
 - Python web framework used to implement the backend logic of the system.
- **Bootstrap Framework:** <mailto:https://getbootstrap.com/>
 - A front-end framework used to design responsive and user-friendly web interfaces.
- **IEEE SRS Standard:** mailto:https://dspmuranchi.ac.in/pdf/Blog/srs_template-ieee.pdf
 - Guidelines for writing clear and structured software requirement specifications.
- **Entity Relationship Diagram (ERD):** <mailto:https://www.drawio.com/>

- A modeling technique used to represent the database structure and relationships between entities.

2. Overall Description

2.1 Product Perspective

- The Smart Ride Fare Comparison System is a new, independent web application that allows users to compare ride fares from Uber, Didi, and On Drive.
- Users interact with the system via web browsers on desktop or mobile devices.
- The system retrieves fare data from mock data and provides redirection links to the ride-hailing apps for booking. Its main components are:
 - **Frontend** (HTML, CSS, Bootstrap) for user input and price display.
 - **Backend** (Django) for data processing and management.
 - **Database** for storing ride information.
 - **Figma** for make initial design.
- Interfaces include the user interface, data source, and external applications, ensuring smooth interaction and clear separation of system components.

2.2 Product Functions

The Smart Ride Fare Comparison System provides the following main functions:

- ❖ Login
- ❖ Register
- ❖ Pick-Up Location Input
- ❖ Drop-Off Location Input
- ❖ Price Comparison Display
- ❖ Ride Selection & Redirection

2.3 User Classes and Characteristics

Customers:

- People who use the system to compare ride fares and select a preferred ride.

- Interact with all main functions: registration, login, pick-up and drop-off input, fare comparison, and ride selection.
- Technical expertise: Low; interface should be simple, intuitive, and user-friendly.
- Importance: The system is designed entirely around the needs of these users.

2.4 Operating Environment

- ❖ Operating System supports all known operating systems, such as Windows, Linux, Android.
- ❖ The system supports modern web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

2.5 Design and Implementation Constraints

- Must use mock data for ride prices because real APIs are unavailable.
- Internet connection is required for price comparison and app redirection.
- User data must be handled safely, especially login credentials
- System must support standard web browsers and be responsive for different devices.

3. External Interface Requirements

3.1 User Interfaces

The Go Cheap system is a web-based application that provides an interactive and user-friendly interface accessible through modern web browsers. The user interface is designed using Figma and implemented using HTML, CSS, JavaScript, and Bootstrap to ensure responsiveness and ease of use.

The main user interfaces include:

- **Registration and Login Interface:** Allows users to create an account using their email address and securely log in to the system.
- **Home Page Interface:** Enables users to enter or automatically detect their current location and manually enter the destination.
- **Ride Comparison Interface:** Displays ride options retrieved from Uber, inDrive, and DiDi, including trip price, and trip time.

- **Ride Selection and Redirection Interface:** Allows the user to select a preferred ride and be redirected to the selected provider's application or website to complete the booking process.
- **Price History Interface:** Displays previously recorded prices and the corresponding ride-hailing company for reference.

The interface follows standard web usability guidelines, including clear navigation, consistent layout, readable fonts, and informative error messages. All screens are fully responsive and support different device sizes.

3.2 Hardware Interfaces

The Go Cheap system is a software-only web application and does not directly interact with any specialized hardware components. It runs on standard user devices such as desktop computers, laptops, tablets, and smartphones.

The hardware requirements are limited to:

- A device capable of running a modern web browser.
- Internet connectivity.

Optional access to the device's location services (GPS) may be used through the web browser to automatically detect the user's current location. No dedicated or external hardware interfaces are required.

3.3 Software Interfaces

The Go Cheap system interacts with several internal software components:

- **Backend Framework:** Django handles server-side logic, business rules, user authentication, and trip comparison requests.
- **Database Management System:** Microsoft SQL Server stores user accounts, authentication data, simulated trip data, price history, and user preferences.
- **Frontend Technologies:** HTML, CSS, JavaScript, and Bootstrap create a responsive and user-friendly web interface.
- **Operating System:** Compatible with Windows, Linux, and macOS.

- Web Browser: Supports modern browsers, including Chrome, Firefox, and Edge.

3.4 Communications Interfaces

The Go Cheap system relies on internet-based communication interfaces to ensure reliable and secure data exchange:

- HTTP/HTTPS Protocols: Used for communication between the client-side web application and the Django backend server.
- Mock API Communication: During the academic phase, simulated API endpoints are used to exchange test data mimicking real ride-hailing service responses.
- Future API Integration: The system is designed to allow seamless replacement of mock APIs with real external service APIs without major architectural changes.
- Security: HTTPS/SSL encryption is applied to protect user data and system communication.

This design ensures reliable testing in an academic environment while remaining scalable for real-world deployment.

4. System Features

4.1 Price Comparator

Description and Priority:

- Compares ride prices across Uber, inDrive, and DiDi to provide users with the most suitable ride based on affordability and vehicle quality.
- **Priority:** High

Stimulus/Response Sequences:

1. User enters pick-up and drop-off locations.
2. System fetches ride prices and vehicle details from internal or mock APIs.
3. System displays a sorted list highlighting the most suitable rides.

Functional Requirements:

- **REQ-1:** The system shall retrieve ride prices from all supported services.
- **REQ-2:** The system shall compare prices and highlight the cheapest option considering vehicle quality.
- **REQ-3:** The system shall allow the user to select a ride for further action.

4.2 User Interface

Description and Priority:

- Provides a clear, consistent, and responsive web interface for interacting with the system.
- **Priority:** High

Stimulus/Response Sequences:

1. User navigates the website.
2. System displays interactive forms and ride comparison results.
3. User interacts with buttons, inputs, and navigation elements.

Functional Requirements:

- **REQ-4:** The system shall provide a responsive web interface compatible with desktop and mobile devices.
- **REQ-5:** The system shall highlight main buttons and important text using visual cues.
- **REQ-6:** The system shall use consistent layout and color schemes across all pages.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The Go Cheap system shall provide fast and responsive performance to ensure an efficient user experience when comparing ride prices from different transportation services.

- The system shall display price comparison results within **2–3 seconds** after the user submits trip details such as pickup and destination.

- Requests to external ride service APIs (Uber, Didi, and InDrive) shall be handled **asynchronously** to prevent delays or blocking in the user interface.

5.2 Safety Requirements

The Go Cheap system shall be designed to prevent misuse and ensure safe operation.

- The system shall protect user data and ensure secure data transmission.
- The system shall display disclaimers indicating that displayed prices are estimates and may vary.
- The system shall restrict access to administrative features and grant full control only to the website owner through secure authentication.

5.3 Security Requirements

The Go Cheap system shall ensure that all data is safe from unauthorized use or breaches.

- Only verified users shall be able to access restricted features.
- Administrative functions shall be accessible solely by the website owner.
- Sensitive data shall be encrypted during storage and transmission.

5.4 Software Quality Attributes

The Go Cheap system shall be:

- **Usable:** easy to navigate for quick price comparison.
- **Reliable:** maintain high uptime and handle errors safely.
- **Secure:** protect user data and limit admin access.
- **Portable:** work across browsers and devices.
- **Flexible:** allow future integration with new ride services.

5.5 Business Rules

- Only registered users can save preferences or queries.
- Only the website owner can access administrative functions.

- Displayed ride prices are estimates and may vary.
- User data must not be shared without consent.
- Only verified ride services are integrated.

6. Other Requirements

Appendix A: Glossary

API: Application Programming Interface used to fetch prices from Uber, Didi, InDrive.

Appendix B: Analysis Models

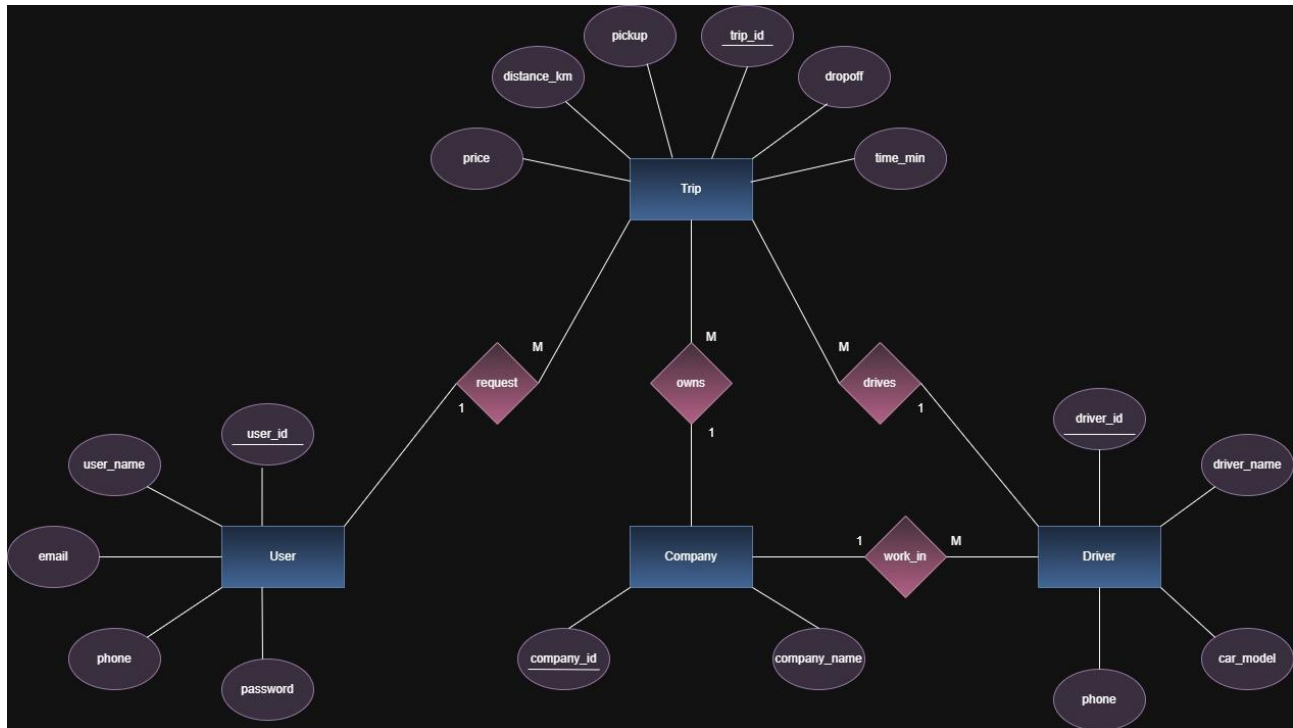
Users submit trip details → System fetches prices from APIs → System displays comparison results.

Appendix C: To Be Determined List

- Exact API rate limits for each ride service.
- Pricing update frequency from external services.
- Future language support beyond English.

7. Database Diagram

7.1 ERD Diagram

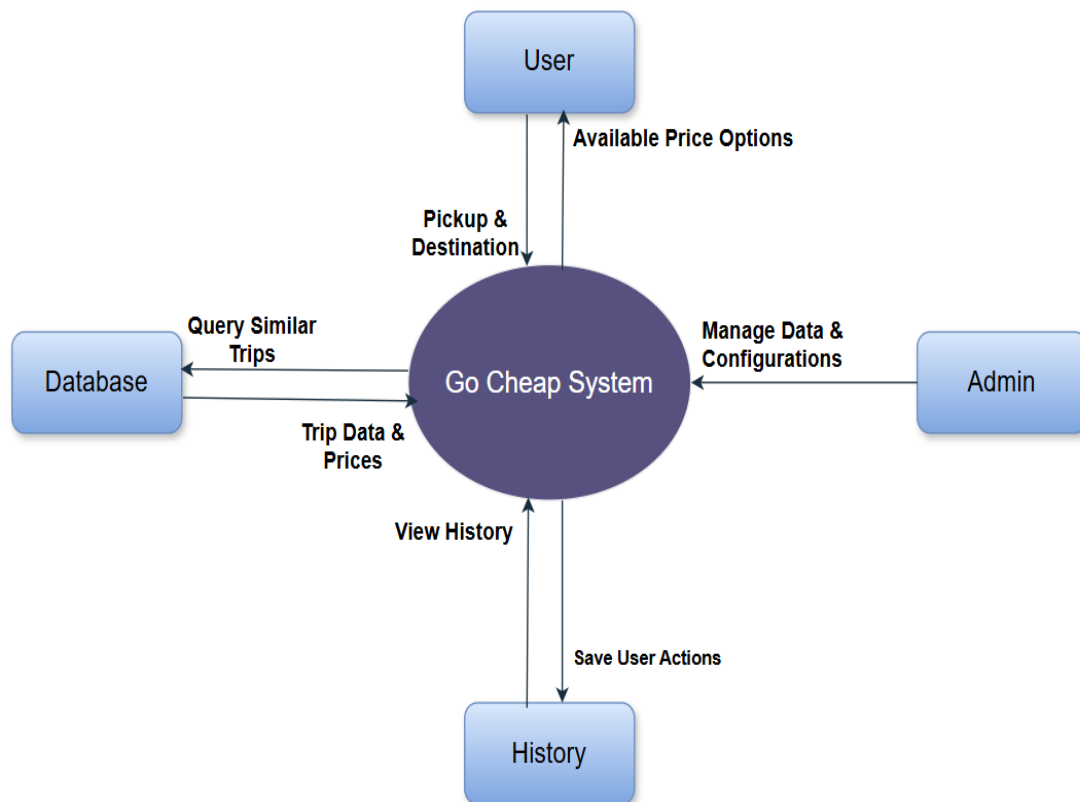


7.2 Relation Schema Diagram

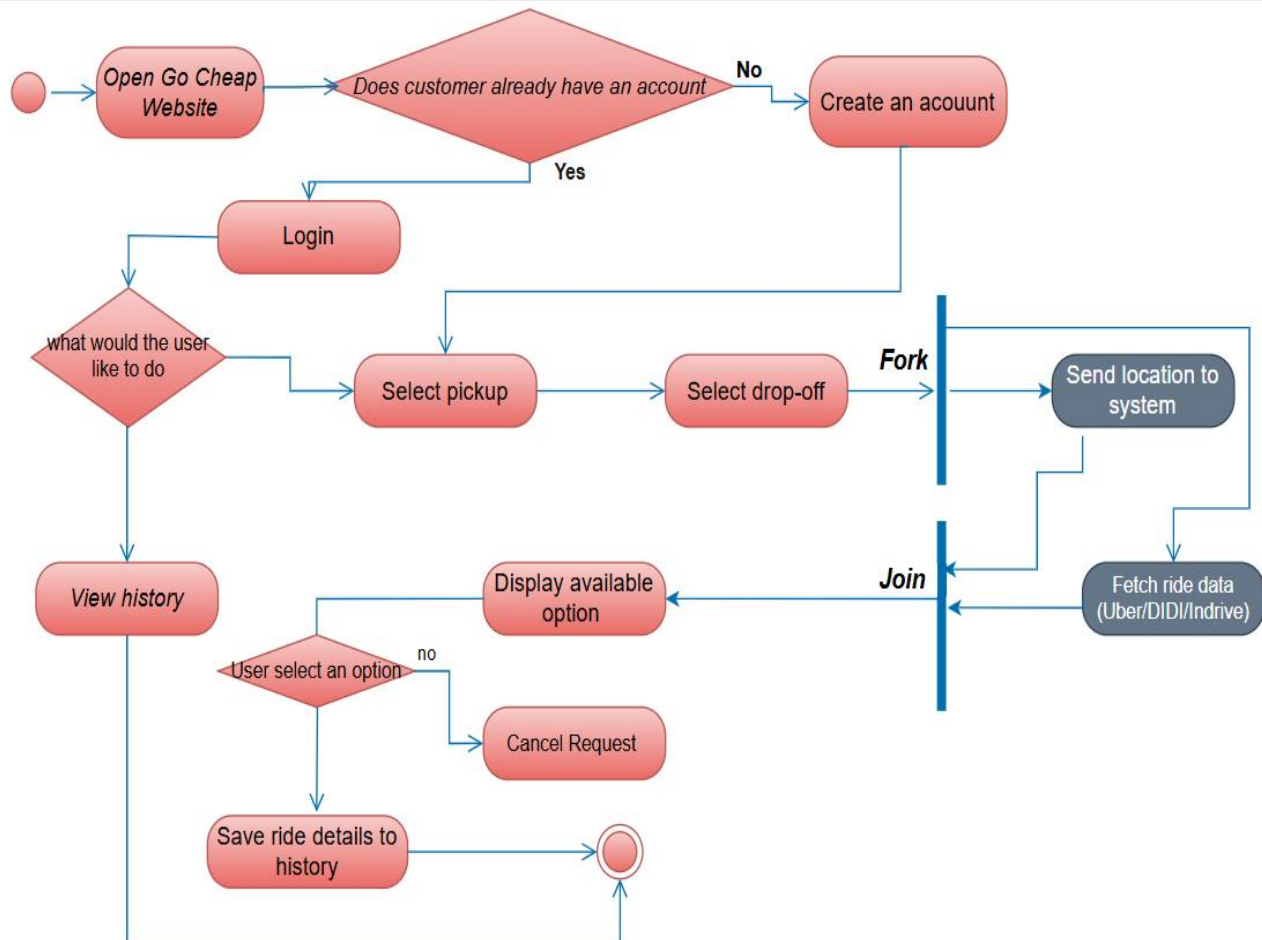


8. System modeling

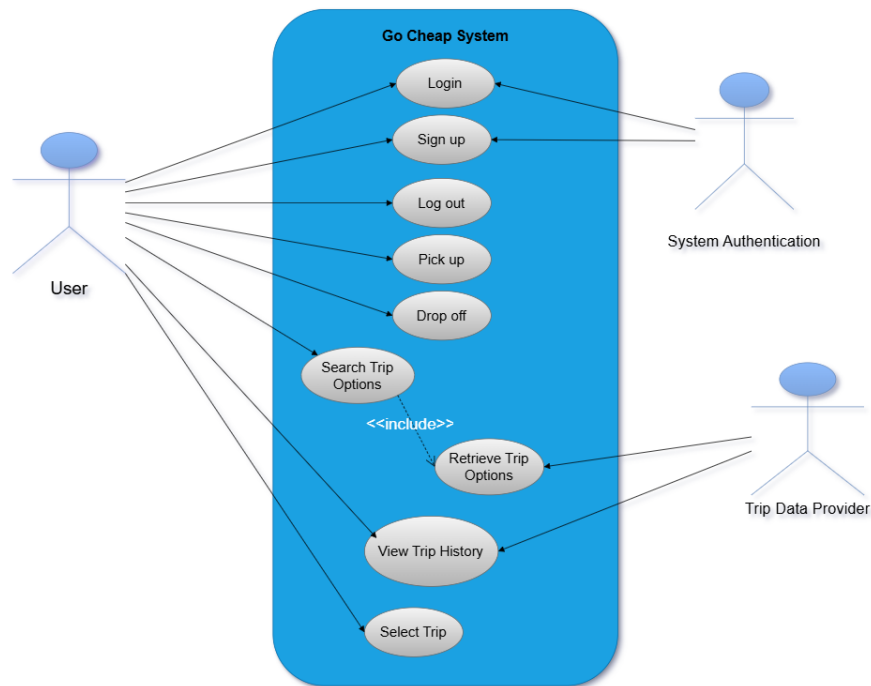
8.1 Context Diagram



8.2 Process perspective Diagram



8.3 Use case Diagram



8.4 Tables of use cases

1. Sign up

Use case	Sign up
Actors	User, System Authentication.
Description	The user creates a new account in the system by entering personal and login information (name, email, password).
Data	User profile data (name, email, phone number), encrypted password.
Stimulus	The user selects the "Sign up" option from the system interface.
Response	the user is redirected to the home page.
Comments	The system must verify that the email is unique and that the password meets security requirements (length).

2. Login

Use case	Login
Actors	User, System Authentication.
Description	The user logs into the system by providing valid login credentials (email and password).
Data	Email, password, stored user credentials in the database.
Stimulus	The user enters login credentials and presses the "Login" button.
Response	If credentials are valid, the user session is created and the main interface is displayed; if invalid, an error message is shown.
Comments	The system must store passwords securely.

3. Search Trip Options

Use case	Search Trip Options
Actors	User.
Description	The user searches for available trips by specifying origin, destination. The system queries the Trip Data Provider and displays matching trip options.
Data	Search parameters (origin, destination), trip details (distance in km, driver name , car model , phone ,price, duration, company name).
Stimulus	The user enters search criteria and presses the "Search" button.
Response	A list of available trip options that match the criteria is displayed with relevant details and filters applied.
Comments	Filters such as price, company be applied.

4. Retrieve Trip Options

Use case	Retrieve Trip Options
Actors	Trip Data Provider
Description	The system sends the user's search parameters to the Trip Data Provider and retrieves the available trips matching those parameters.
Data	Search parameters, trip options returned by the provider.
Stimulus	An internal call from the "Search Trip Options" use case.
Response	A set of trip options is returned to the Go Cheap System to display to the user.
Comments	Network failures or provider unavailability must be handled with proper error messages and fallback mechanisms.

5. Select Trip

Use case	Select Trip
Actors	User.
Description	The user selects one trip from the list of available search results to direct him/her to certain app for booking trip.
Data	Selected trip identifier.
Stimulus	The user clicks on a trip from the search results list.
Response	System direct user to selected app to book certain trip.
Comments	The system should verify in real-time that the selected trip is still available.

6. Pick up

Use case	Pick up
Actors	User.
Description	The user specifies the pick-up location (starting point) for the chosen trip during the booking process.
Data	Pick-up location (address), route information.
Stimulus	The user chooses the pick-up point in the booking process.
Response	The pick-up location is saved as part of the trip booking information; the system may display the pick-up time and any relevant instructions.
Comments	The system must ensure that the pick-up point is within the supported service area and that the location is accessible and valid.

7. Drop off

Use case	Drop off
Actors	User.
Description	The user specifies the drop-off location (destination point) for the chosen trip during the booking process.
Data	Drop-off location (address), route information.
Stimulus	The user chooses the drop-off point in the booking process.
Response	The drop-off location is saved as part of the trip booking information.
Comments	The system suggest common or valid drop-off points near the destination to assist the user in making a quick selection.

8. View Trip History

Use case	View Trip History
Actors	User.
Description	The user views a list of previous and current trips associated with their account, including booking details.
Data	Past trip records (distance in km, driver name , car model , phone ,price, duration, company name).
Stimulus	The user selects the "View Trip History" option from their account dashboard.
Response	A list of past trips is displayed with filtering options.
Comments	The system should display only trips relevant to the logged-in user for privacy and security.

9. Log out

Use case	Log out
Actors	User.
Description	The user ends the current session and exits the system securely, returning to the public sign up page.
Data	User session data (session token, data).
Stimulus	The user clicks the "Log out" option from the menu.
Response	The session is terminated, and the user is redirected to the sign up page.
Comments	Important for security.

8.5 Class Diagram

